



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G4

System Integration and Architecture

System Design Document (SDD)

Project Title: Pawfolio

Prepared By: Arong, Kylene Jacaban

Version: 2.0

Date: 03/03/2026

Status: Draft

REVISION HISTORY TABLE

Versio n	Date	Author	Changes Made	Status
1.0	18/02/2026	Kylene J. Arong	Initial draft	Draft
1.1	28/02/2026	Kylene J. Arong	Revised initial draft	Revised
2.0	03/03/2026	Kylene J. Arong	Final draft for submission	Final draft
	[Date]	[Your Name]	Added UI/UX designs	Review
	[Date]	[Your Name]	Incorporated feedback	Revised
	[Date]	[Your Name]	Final review and corrections	Final
	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	5
3.0 NON-FUNCTIONAL REQUIREMENTS	9
4.0 SYSTEM ARCHITECTURE	11
5.0 API CONTRACT & COMMUNICATION	12
6.0 DATABASE DESIGN	17
7.0 UI/UX DESIGN	18
8.0 PLAN	21

EXECUTIVE SUMMARY

1.1 Project Overview

Pawfolio is a pet health record management application that allows pet owners to securely store, manage, and track their pets' medical information. The system consists of a Spring Boot backend API, a React web application, and an Android mobile application, all integrated to provide a consistent and accessible experience across platforms.

1.2 Objectives

1. Develop a secure pet health record management system with user authentication and authorization.
2. Implement JWT-based authentication with BCrypt password hashing.
3. Enforce Role-Based Access Control with USER and ADMIN roles.
4. Build a three-tier architecture using Spring Boot (backend), React (web), and Android Kotlin (mobile).
5. Implement full CRUD operations for the core business module (Pet and Health Records).
6. Integrate Google OAuth login and generate JWT after successful authentication.
7. Consume a public external API and display retrieved data within the system.
8. Implement secure file upload functionality for medical documents.
9. Send system-generated emails using SMTP.
10. Ensure proper RESTful API design, validation, and secure data handling.

1.3 Scope

Included Features:

- User registration and authentication (email/password)
- Google OAuth login
- JWT authentication with protected routes
- /auth/me endpoint
- Role-Based Access Control (User and Admin)
- User profile management
- Pet profile creation and management (CRUD)
- Pet health record management (CRUD)
- File upload for medical documents (image/pdf)

- External API integration (public pet breed API)
- Email notifications (welcome email and record notification)
- Dashboard overview of pets and recent records
- Responsive web interface (React)
- Native Android mobile application (Kotlin, XML layouts, Retrofit)
- Relational database using MySQL or PostgreSQL
- Layered backend architecture (Controller, Service, Repository, DTO, Security, Global Exception Handler)

Excluded Features:

- Online veterinary appointment booking
- Payment gateway integration
- Push notifications
- Integration with external veterinary hospital systems
- Advanced analytics or reporting
- Multi-language support

1.0 INTRODUCTION

1.1 Purpose

This document provides the comprehensive Software Design Description (SDD) for the Pawfolio system. It defines the system architecture, functional and non-functional requirements, API design, and implementation scope to ensure seamless integration between backend, web, and mobile components.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: Pawfolio

Domain: Pet Care / Health Management

Primary Users: Pet Owners (User Role), System Administrators (Admin role)

Problem Statement: Pet owners often struggle to keep track of their pets' medical records across clinics and physical documents.

Solution: Pawfolio provides a centralized digital platform for storing and managing pet health records, accessible anytime through web or mobile applications.

2.2 Core User Journeys

Journey 1: User Registration & Authentication

1. User opens the web or mobile application.
2. User registers using email/password or Google OAuth.
3. Password is hashed using BCrypt.
4. System generates JWT upon successful login.
5. User accesses protected routes.
6. User retrieves authenticated profile via [/auth/me](#).

Journey 2: Pet Profile Management

1. User logs in to the web or mobile app.
2. User creates a pet profile.
3. User views pet details.
4. User updates pet information.
5. User deletes a pet profile.
6. System validates ownership before modification.

Journey 3: Health Record Management

1. User selects a pet.
2. User views health record history.
3. User adds a new health record.
4. User uploads a supporting medical document.
5. User edits or deletes existing records.
6. System validates data and ownership.

Journey 4: Administrator Management

1. Admin logs in with an Admin role.
2. Admin accesses admin-protected endpoints.
3. Admin views all users.

4. Admin views all pets and records.
5. Admin deletes inappropriate data.

Journey 5: System Integration Features

1. User registers and receives a welcome email.
2. User adds a health record and receives notification email.
3. System fetches breed information from external API.
4. Retrieved API data is displayed in pet profile.
5. File upload is stored and linked to database.

Journey 6: User Profile Management

1. User logs in.
2. User navigates to Profile page.
3. User updates personal information (e.g., name).
4. User saves changes.
5. System validates input.
6. Database is updated.
7. Updated information is reflected in the dashboard.

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (register, login, logout)
2. JWT authentication
3. BCrypt password hashing
4. Protected routes/pages
5. [/auth/me](#) endpoint
6. Google OAuth login
7. Role-Based Access Control (minimum 2 roles)
8. Pet profile CRUD operations
9. Pet health record CRUD operations
10. File upload functionality
11. External API integration
12. Email sending via SMTP

13. Proper validation for all inputs
14. React web app consuming backend API
15. Android mobile app consuming same backend API

SHOULD HAVE

1. Categorization of health records (vaccination, medication, checkup)
2. Search or filter health records by date or type
3. Input validation feedback messages
4. Responsive UI design across screen sizes

COULD HAVE

1. Health record summaries per pet
2. Basic reminder notes (non-notification based)

WON'T HAVE

1. Payment gateway integration
2. Vet appointment scheduling
3. Push notifications
4. Advanced analytics
5. Multi-language support

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Google Login, Forgot Password
- **Fields:** Email, Password, Confirm Password
- **Validation:** Email format, password strength, uniqueness
- **API Endpoints:**
 - POST /auth/register, POST /auth/login, POST /auth/oauth/google, POST /auth/logout, GET /auth/me
- **Security:** JWT authentication, bcrypt password hashing, Authorization header required for protected routes, Role-based authorization using Spring Security, DTO usage (no password exposure in responses)

Feature: Pet Profile Management

- **Screens:** Dashboard, Add/Edit Pet
- **Fields:** Pet name, species, breed, age
- **Functions:** Add, edit, delete pet, view pet details
- **API Endpoints:** GET /pets, POST /pets, PUT /pets/{id}, DELETE /pets/{id}
- **Access Control:** Users can only manage their own pets, Admin can view and manage all pets.

Feature: Pet Health Record Management

- **Screens:** Pet Detail, Add/Edit Health Record
- **Record Types:** Vaccination, medication, checkup
- **Fields:** Record type, description, date, notes, File attachment (optional)
- **Functions:** Add, view, update, delete health records
- **Persistence:** Records stored per pet per user
- **API Endpoints:** GET /pets/{petId}/records, POST /pets/{petId}/records, PUT /records/{id}, DELETE /records/{id}
- **File Upload:** Accept image or PDF, File stored on server, File path saved in database, File linked to health record, User can view/download file

Feature: External API Integration

- System consumes a public pet breed API.
- When a pet is created or viewed, breed information is retrieved.
- Retrieved data is displayed in the pet profile.

Feature: Email Notifications

- **Emails sent:** Welcome email upon successful registration, Notification email when a new health record is added.
- **Implementation:** SMTP configuration, Proper error handling

2.5 Acceptance Criteria

AC-1: Successful User Registration

- **Given** I am a new user
- **When** I enter a valid email and strong password
- **And** confirm that the password matches

- **And** click "Create Account"
- **Then** my account should be created
- **And** my password should be securely hashed
- **And** I should be automatically logged in
- **And** redirected to the dashboard

AC-2: Google OAuth Login

- **Given** I choose to log in using Google
- **When** authentication is successful
- **Then** my account should be created or linked in the database
- **And** the system should generate its own JWT token
- **And** I should be redirected to the dashboard

AC-3: Pet Profile Management

- **Given** I am logged in as a pet owner
- **When** I create a new pet with valid details
- **And** save the pet profile
- **Then** the pet should be stored in the database
- **And** displayed in my dashboard
- **When** I edit or delete my pet profile
- **Then** the changes should be reflected immediately

AC-4: Pet Health Record Management with File Upload

- **Given** I am logged in
- **And** I have an existing pet profile
- **When** I add a new health record with valid details
- **And** upload a medical document
- **And** save the record
- **Then** the record should be stored in the database
- **And** the file should be saved on the server
- **And** the record should appear in the pet's medical history

AC-5: Role Based Access Control

- **Given** I am logged in as a regular user
- **When** I attempt to access another user's pet data
- **Then** access should be denied

- **Given** I am logged in as an administrator
- **When** I access system-wide pet records

- **Then** I should be authorized to view and manage them

AC-6: Email Notification

- **Given** I successfully register
- **Then** I should receive a welcome email
- **Given** I successfully add a health record
- **Then** I should receive a notification email confirming the record update

AC-7: External API Integration

- **Given** I create or view a pet profile
- **When** breed information is requested
- **Then** the system should retrieve breed data from a public API
- **And** display the retrieved breed details in the pet profile

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time ≤ 2 seconds for 95% of requests under normal load.
- Dashboard load time ≤ 3 seconds on standard broadband connection.
- Mobile application cold start time ≤ 3 seconds.
- System must support at least 100 concurrent authenticated users.
- Database queries should complete within 500 milliseconds for standard CRUD operations.
- File upload processing time ≤ 5 seconds for files up to 5MB.

3.2 Security Requirements

- HTTPS must be enforced for all communications.
- JWT-based authentication must protect all secured endpoints.
- Passwords must be hashed using BCrypt (minimum 12 salt rounds).
- Protected routes must validate JWT tokens before granting access.
- Role-Based Access Control must restrict endpoints based on USER and ADMIN roles.

- SQL injection must be prevented through parameterized queries (JPA/Hibernate).
- Cross-Site Scripting (XSS) protection must be implemented on web forms.
- File uploads must validate file type and size before storage.
- Password fields must never be exposed in API responses (DTO usage required).

3.3 Compatibility Requirements

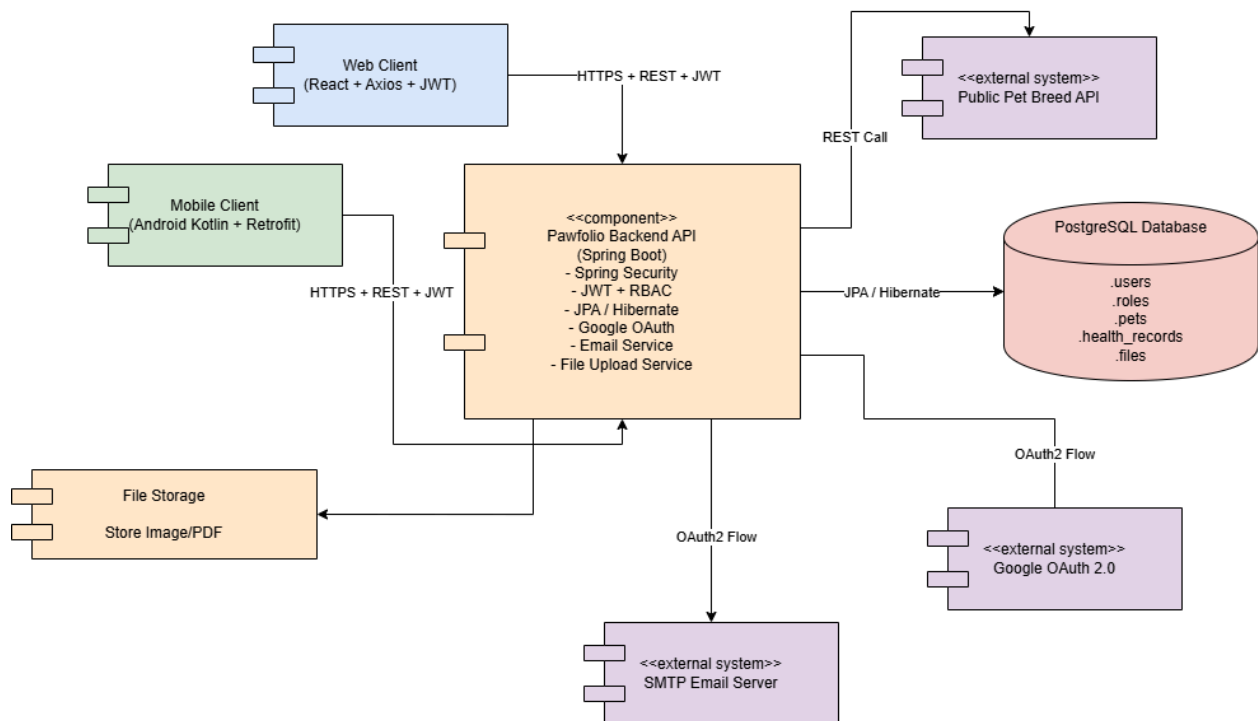
- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** API Level 34+ (Android 14+)
- **Screen Sizes:** Mobile(360px+), tablet(768px+), desktop(1024px+)
- **Operating Systems:** Windows, macOS, Linux

3.4 Usability Requirements

- New users should be able to register and create a pet profile within 5 minutes.
- Navigation must remain consistent across all pages.
- Error messages must clearly describe validation issues.
- Forms must provide immediate validation feedback.
- Mobile UI must use appropriately sized touch targets (minimum 44x44px).
- Role-based UI elements must dynamically adjust based on logged-in user role.
- Uploaded files must be easily accessible through a clear “View/Download” option.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, XML-Based Layouts, Retrofit
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

- **Base URL:** `https://api.pawfolio.com/api/v1`
- **Format:** JSON for all requests/responses
- **Authentication:** Bearer token (JWT) in Authorization header

- **Response Structure:**

json

```
{  
  "success": boolean,  
  "data": object | null,  
  "error": {  
    "code": string,  
    "message": string,  
    "details": object | null  
  },  
  "timestamp": string  
}
```

5.2 Endpoint Specifications

Authentication Endpoints

POST /auth/register

Body: {email, password, confirmPassword, fullName}

Response: {user: {id, email, fullName, role}, token, refreshToken}

POST /auth/login

Body: {email, password}

Response: {user: {id, email, fullName, role}, token, refreshToken}

POST /auth/google

Body: {authorizationCode}

Response: {user: {id, email, fullName, role}, token, refreshToken}

POST /auth/logout

Headers: Authorization: Bearer {token}

Response: {message: "Logged out successfully"}

GET /auth/me

Headers: Authorization: Bearer {token}

Response: {user: {id, email, fullName, role}}

User Endpoints

PUT /users/profile (Authenticated)

Body: {fullName}

Response: {user: {id, email, fullName, role}}

Pet Endpoints

GET /pets (Authenticated)

Response: {pets: [{id, name, species, breed, age, ownerId}]}

GET /pets/{id} (Authenticated)

Response: {pet: {id, name, species, breed, age, ownerId}}

POST /pets (Authenticated)

Body: {name, species, breed, age}

Response: {pet: {...}}

PUT /pets/{id} (Owner or Admin)

Body: {name, species, breed, age}

Response: {pet: {...}}

DELETE /pets/{id} (Owner or Admin)

Response: {message: "Pet deleted successfully"}

Pet Health Records Endpoints

GET /pets/{petId}/records (Authenticated)

Response: {records: [{id, type, description, recordDate, fileUrl}]}

POST /pets/{petId}/records (Authenticated)

Body: {type, description, recordDate}

File: multipart/form-data (image/pdf optional)

Response: {record: {...}}

PUT /records/{id} (Owner or Admin)

Body: {type?, description?, recordDate?}

Response: {record: {...}}

DELETE /records/{id} (Owner or Admin)

Response: {message: "Health record deleted successfully"}

External API Integration

GET /pets/breeds/search?name=labrador

Response: {breeds: [{name, temperament, lifeSpan, origin}]}

Admin Endpoints

GET /admin/users (Admin only)

Response: {users: [...]}

GET /admin/pets (Admin only)

Response: {pets: [...]}

5.3 Error Handling

HTTP Status Codes

- 200 OK – Successful request
- 201 Created – Resource created
- 400 Bad Request – Invalid input
- 401 Unauthorized – Authentication required or failed
- 403 Forbidden – Insufficient permissions
- 404 Not Found – Resource does not exist
- 409 Conflict – Duplicate resource
- 500 Internal Server Error – Server error

Error Code Examples

json

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "AUTH-001",
    "message": "Invalid credentials",
    "details": "Email or password is incorrect"
  },
  "timestamp": "2026-03-03T10:30:00Z"
```

```
}

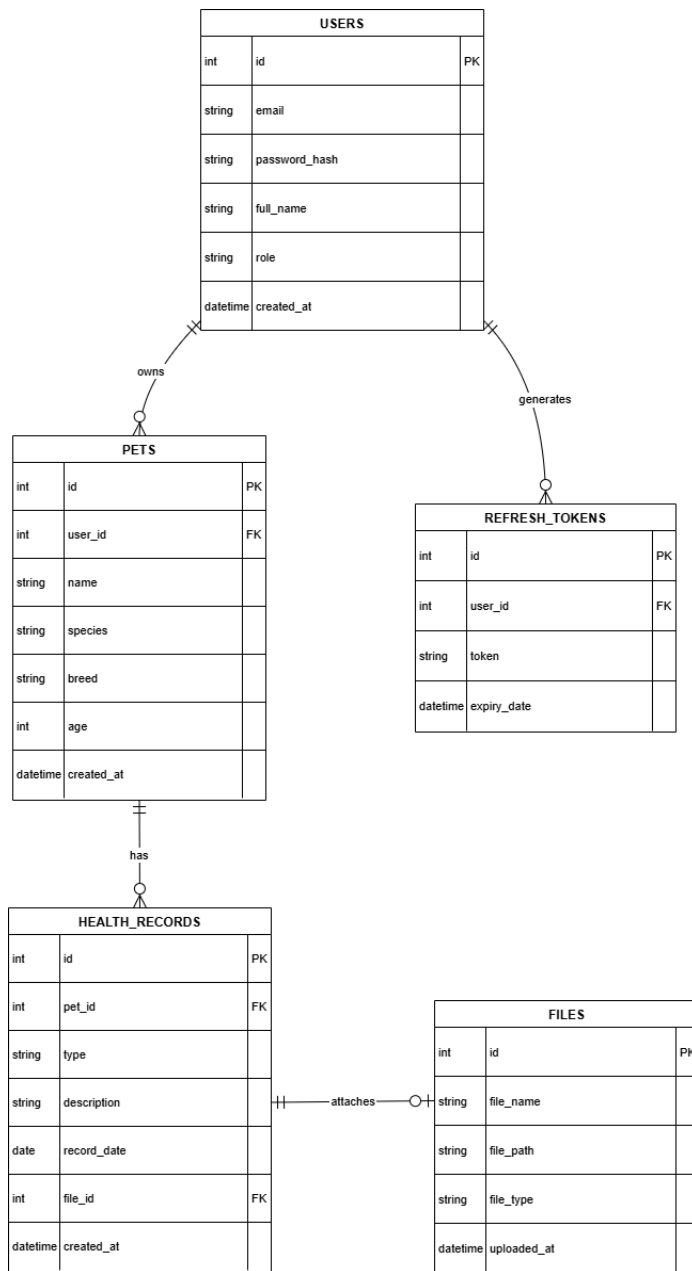
{
  "success": false,
  "data": null,
  "error": {
    "code": "VALID-001",
    "message": "Validation failed",
    "details": {
      "email": "Email is required",
      "password": "Must be at least 8 characters"
    }
  },
  "timestamp": "2026-03-03T10:30:00Z"
}
```

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Unauthorized pet access
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **One-to-Many:** User → Pets (Each user can manage multiple pets)
- **One-to-Many:** Pet → HealthRecords (Each pet can have multiple health records)
- **One-to-One (Optional):** HealthRecord ↔ File (A health record may have one associated uploaded file)
- **One-to-Many:** User → RefreshTokens (A user can have multiple active refresh tokens)

- **Many-to-One:** Pet → User (Each pet belongs to exactly one user)
- **Many-to-One:** HealthRecord → Pet (Each health record belongs to one pet)
- **Many-to-One:** File → HealthRecord (Each file is linked to one health record)

Key Tables:

1. **users** – User accounts and authentication data
2. **pets** – Pet profile information
3. **health_records** – Medical records for pets
4. **files** – Uploaded medical documents
5. **refresh_tokens** – JWT refresh tokens

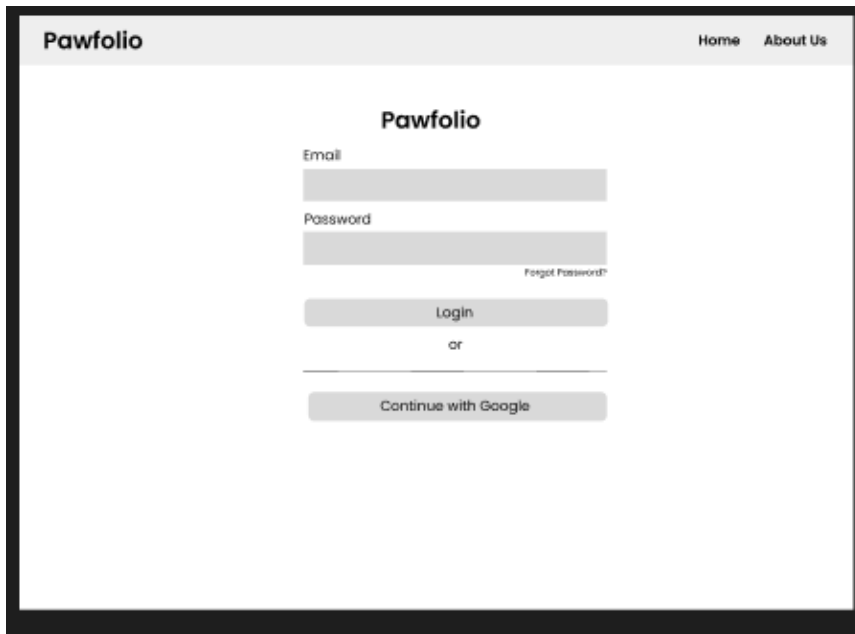
Table Structure Summary:

- **users:** id, email, password_hash, full_name, role, created_at
- **pets:** id, user_id, name, species, breed, age, created_at
- **health_records:** id, pet_id, type, description, record_date, file_id (nullable), created_at
- **files:** id, file_name, file_path, file_type, uploaded_at
- **refresh_tokens:** id, user_id, token, expiry_date

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Login Page



The login page features a header with the 'Pawfolio' logo on the left and 'Home' and 'About Us' links on the right. The main content area is centered and contains the 'Pawfolio' title. Below the title are input fields for 'Email' and 'Password'. A 'Forgot Password?' link is positioned to the right of the password field. A 'Login' button is centered below the password field. Below the login button is the text 'or' followed by a horizontal line. At the bottom of the form is a 'Continue with Google' button.

Pawfolio

Home About Us

Pawfolio

Email

Password

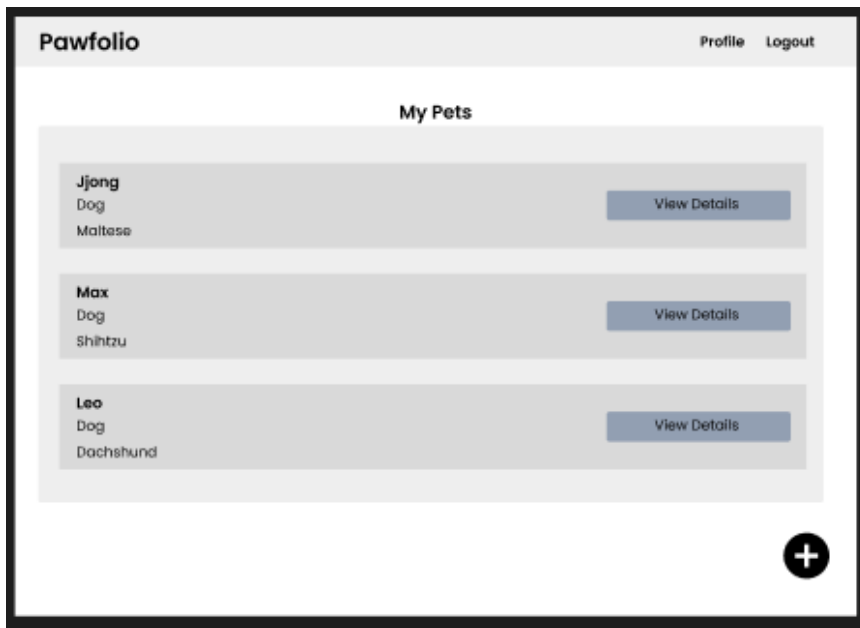
[Forgot Password?](#)

Login

or

[Continue with Google](#)

Dashboard



The dashboard has a header with the 'Pawfolio' logo on the left and 'Profile' and 'Logout' links on the right. The main content area is titled 'My Pets' and contains a list of three pets. Each pet entry includes the pet's name, species, breed, and a 'View Details' button. At the bottom right of the dashboard is a circular button with a plus sign.

Pawfolio

Profile Logout

My Pets

Jjong	Dog	Maltese	View Details
Max	Dog	Shih-tzu	View Details
Leo	Dog	Dachshund	View Details

+

View Pet Details Page

Pawfolio

ProfileLogout

Pet Details

Jjong

Dog

Maltese

13 years old

Health Records

Vaccination

12/02/2025

Anti-rabies

View

Vaccination

12/02/2025

Anti-rabies

View

Add Pet Page

Pawfolio

ProfileLogout

←

Add Pet

Pet Name

Species

Breed

Age

Save

Add Pet Health Record Page

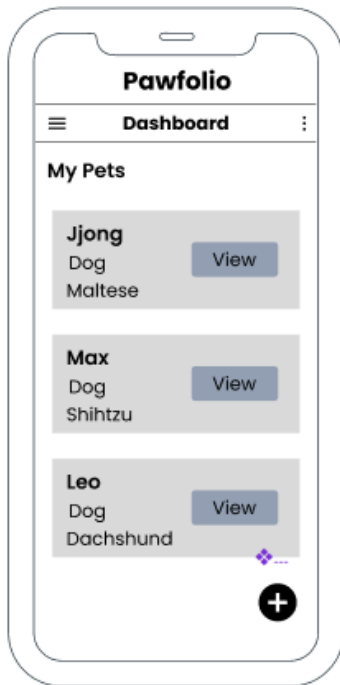
The wireframe shows a mobile application interface for adding a pet health record. At the top, there is a header bar with the app name "Pawfolio" on the left and "Profile" and "Logout" links on the right. Below the header, a modal form titled "Add Pet Health Record" is displayed. The form includes a back arrow, a "Record Type" dropdown menu, a "Description" text area, a "Record Date" field with a calendar icon, an "Upload File" section with a "Choose file" button and "No file chosen" text, and a "Save" button at the bottom.

7.2 Mobile Application Wireframes

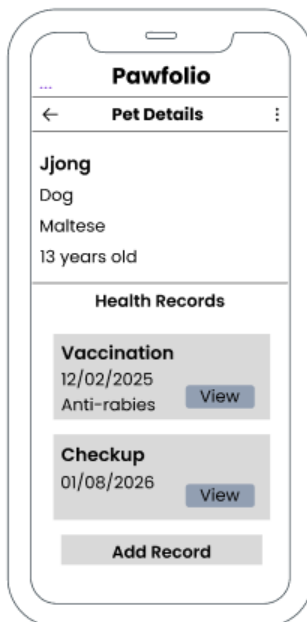
Login Screen

The wireframe shows a mobile application login screen. At the top, there is a placeholder for a profile picture and the app name "Pawfolio". Below this, there are input fields for "Email" and "Password". A "Forgot Password?" link is positioned next to the password field. A "Login" button is located below the password field. A horizontal line separates the login section from the social login section, which includes the text "or" and a "Continue with Google" button.

Dashboard



View Pet Details Screen



Add Pet Screen

A mobile app mockup for the 'Add Pet' screen. The app is titled 'Pawfolio'. The screen has a header with a back arrow, the title 'Add Pet', and a menu icon. Below the header, there are four text input fields labeled 'Pet Name', 'Species', 'Breed', and 'Age'. At the bottom of the form is a 'Save' button.

Add Pet Health Record Screen

A mobile app mockup for the 'Add Pet Health Record' screen. The app is titled 'Pawfolio'. The screen has a header with a back arrow, the title 'Add Pet Health Record', and a menu icon. Below the header, there are four form elements: a 'Record Type' dropdown menu, a 'Description' text area, a 'Record Date' date picker, and an 'Upload File' section with a 'Choose file' button and a 'No file chosen' label. At the bottom of the form is a 'Save' button.

Mobile-Specific Features:

- Touch-optimized buttons (minimum 44x44px tap targets)
- Gesture support (pull-to-refresh on Dashboard and Pet Details)
- Secure JWT storage using encrypted shared preferences
- File upload integration (image/pdf picker)
- Bottom navigation for primary actions (Dashboard, Add Pet, Profile)
- Simplified forms with dropdowns and date picker for easier mobile input
- Role-aware UI (Admin options hidden for regular users)

Design System: (To be implemented in actual app/website)

- **Colors:** Primary (#2563EB), Secondary (#14B8A6), Success (#10B981), Error (#EF4444)
- **Typography:** Inter font family, responsive sizing for mobile and desktop
- **Spacing:** 8px grid system
- **Components:** Consistent button styles, inputs, cards, modals, file upload component with preview
- **Responsive:** Mobile-first approach, breakpoints at 640px, 768px, 1024px

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation structure

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification and endpoint planning

Day 3-4: Database design

Day 5-6: UI/UX wireframes (Web + Mobile)

Day 7: Finalize Implementation plan

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with required dependencies

Day 2: Database configuration and entity creation (User, Pet, HealthRecord, File, RefreshToken)

Day 3: JWT authentication and Spring Security Integration

Day 4: Role Based Access Control (User, Admin)

Day 5: User management endpoints

Week 4: Core Features & Integrations

Day 1: Pet CRUD endpoints

Day 2: Health Record CRUD endpoints

Day 3: File upload implementation

Day 4: Google OAuth Integration

Day 5: External API integration and Email (SMTP) setup

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup with TypeScript and routing

Day 2: Authentication pages (login, register, google login)

Day 3: Dashboard and pet listing page

Day 4: Pet details page

Day 5: Add/Edit Pet forms

Week 6: Complete Web Features

Day 1: Health record pet management features

Day 2: File upload integration

Day 3: Role-based UI adjustments

Day 4: Responsive design polish

Day 5: API integration testing and debugging

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens

Day 3: Dashboard and Pet listing

Day 4: Pet details and Health Record display

Day 5: Retrofit service layer integration

Week 8: Complete Mobile App

Day 1: Add/edit pet forms

Day 2: Add Health Record with file upload

Day 3: Role-based UI behaviour

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing

Day 2: Bug fixes and optimization

Day 3: Security review (JWT, RBAC, file validation)

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway/Heroku/Render)

Day 2: Web app deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final system testing

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional with integrations
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Pet and Health Record API (Week 3-4)
3. File Upload Integration (Week 4)
4. Google OAuth Integration (Week 4)
5. Web-Mobile API Integration (Week 6-8)
6. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Implement authentication early to avoid integration delays
- Buffer time allocated during weekends for unexpected delays and bug fixes.

- Test file upload and OAuth integration early
- Keep version-controlled backups of stable builds
- Focus on core functionality before enhancements